KR MANGALAM UNIVERSITY

SECURE CODING AND VULNERABILTIES LAB

(ENSP 351)

NAME – YASH VASHISTH

B.TECH CSE AI/ML SEC-C

ROLL NUMBER - 2301730149

LAB REPORT

Safe and Unsafe Data Type Conversions in C++

Aim

To write and execute a C++ program that converts between data types—int to short, signed to unsigned, float to int, and string to numeric. The experiment demonstrates truncation, sign-conversion, and overflow through an unsafe version, and then applies range checks and safe conversion helpers to reject out-of-range inputs with clear error messages.

1. Vulnerable / Unsafe Version

The following program performs the conversions without validating the source values before conversion.

```
#include <iostream>
#include <string>
using namespace std;

int main() {

    // 1. int to short
    int a = 40000;
    short b = (short)a;

    cout << "INT to SHORT" << endl;
    cout << "Before: " << a << endl;
    cout << "After : " << b << endl;
    cout << endl;


    // 2. signed to unsigned
    int c = -10;
    unsigned int d = (unsigned int)c;

    cout << "SIGNED to UNSIGNED" << endl;
    cout << "Before: " << c << endl;
    cout << "After : " << d << endl;
    cout << endl;


    // 3. float to int
    float e = 12.75;

    int f = (int)e;

    cout << "FLOAT to INT" << endl;
    cout << "Before: " << e << endl;
    cout << "After : " << f << endl;
    cout << endl;


    // 4. string to int
    string text = "12345";

    int number = stoi(text);

    cout << "STRING to INT" << endl;
    cout << "Before: " << text << endl;
    cout << "After : " << number << endl;

    return 0;
}
```

Output of Vulnerable / Unsafe Version

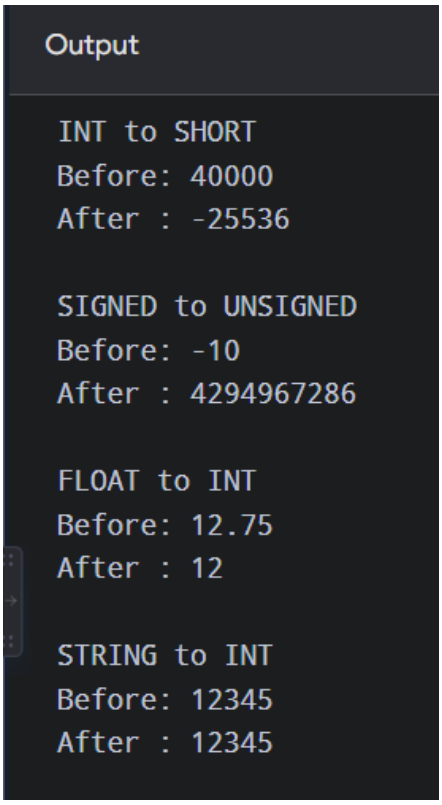

Figure 1: Actual output provided for the vulnerable program.

2. Safe Version

The safe version validates ranges before narrowing or signedness-changing conversions and validates the numeric string before converting it to int. Invalid or out-of-range values are rejected with clear error messages.

```
#include <iostream>
#include <string>
#include <limits>
#include <stdexcept>

using namespace std;


// int to short
short safeIntToShort(int value) {

    if (value < numeric_limits<short>::min() ||
        value > numeric_limits<short>::max()) {

        throw out_of_range("Value is outside the range of short");
    }

    return static_cast<short>(value);
}


// signed int to unsigned int
unsigned int safeSignedToUnsigned(int value) {

    if (value < 0) {
        throw out_of_range(
            "Negative value cannot be safely converted to unsigned"
        );
    }

    return static_cast<unsigned int>(value);
}


// float to int
int safeFloatToInt(float value) {

    if (value < numeric_limits<int>::min() ||
        value > numeric_limits<int>::max()) {

        throw out_of_range("Value is outside the range of int");
    }

    return static_cast<int>(value);
}


// string to int
int safeStringToInt(string text) {

    try {

        size_t position;

        long long value = stoll(text, &position);

        // Check if the complete string was converted
        if (position != text.length()) {
            throw invalid_argument(
                "String contains non-numeric characters"
            );
        }

        // Check int range
        if (value < numeric_limits<int>::min() ||
            value > numeric_limits<int>::max()) {

            throw out_of_range(
                "Number is outside the range of int"
            );
        }

        return static_cast<int>(value);

    }
    catch (const invalid_argument&) {

        throw invalid_argument(
            "Invalid numeric string"
        );

    }
    catch (const out_of_range&) {

        throw out_of_range(
            "String number is too large for int"
        );
    }
}


int main() {

    // 1. int to short
    try {
```

```
        int a = 40000;

        cout << "INT to SHORT" << endl;
        cout << "Before: " << a << endl;

        short b = safeIntToShort(a);

        cout << "After : " << b << endl;
    }
    catch (const exception& error) {
        cout << "ERROR: " << error.what() << endl;
    }

    cout << endl;


    // 2. signed to unsigned
    try {
        int c = -10;

        cout << "SIGNED to UNSIGNED" << endl;
        cout << "Before: " << c << endl;

        unsigned int d = safeSignedToUnsigned(c);

        cout << "After : " << d << endl;
    }
    catch (const exception& error) {
        cout << "ERROR: " << error.what() << endl;
    }

    cout << endl;


    // 3. float to int
    try {
        float e = 12.75;

        cout << "FLOAT to INT" << endl;
        cout << "Before: " << e << endl;

        int f = safeFloatToInt(e);

        cout << "After : " << f << endl;
        cout << "(Decimal part was truncated)" << endl;
    }
    catch (const exception& error) {
        cout << "ERROR: " << error.what() << endl;
    }

    cout << endl;


    // 4. string to int
    try {
        string text = "99999999999999999999999";

        cout << "STRING to INT" << endl;
        cout << "Before: " << text << endl;

        int number = safeStringToInt(text);

        cout << "After : " << number << endl;
    }
    catch (const exception& error) {
        cout << "ERROR: " << error.what() << endl;
    }

    return 0;
}
```

Output of Safe Version

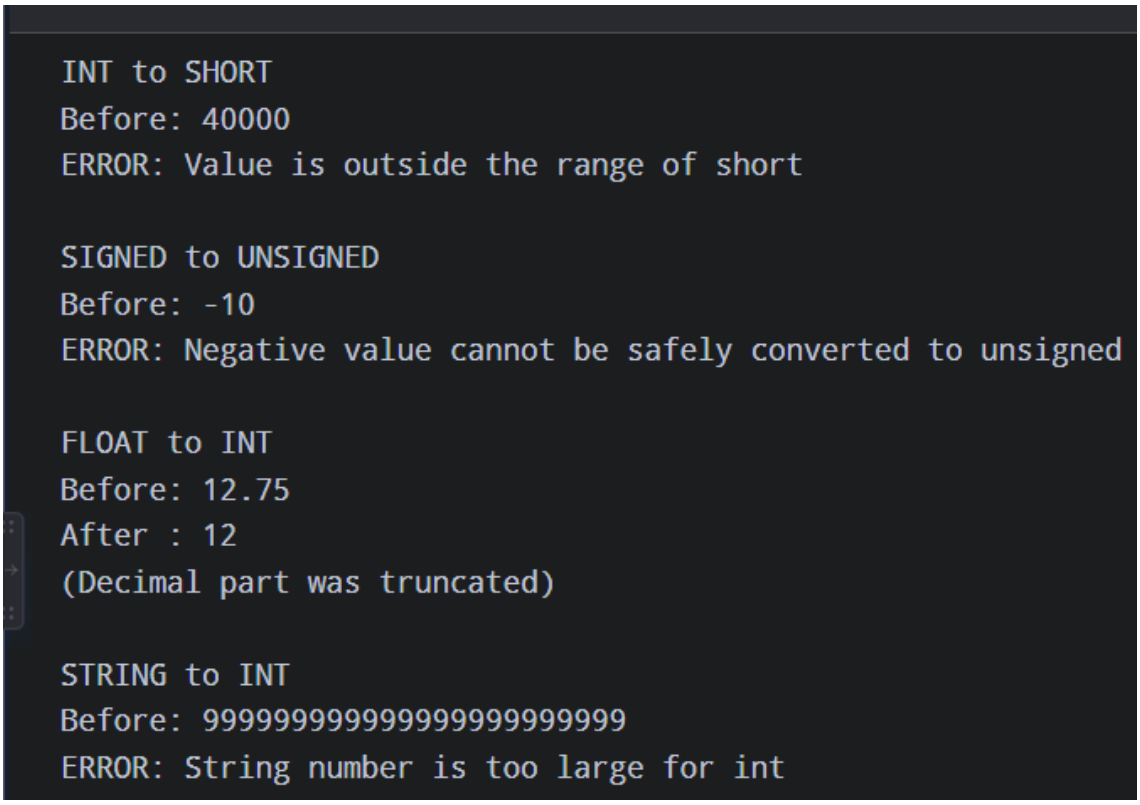

Figure 2: Actual output provided for the safe program.

3. Observations

• **int to short:** 40000 produced -25536 in the submitted unsafe run, demonstrating an unexpected narrowing result. The safe version rejected the value because it was outside the range of short.

• **signed to unsigned:** -10 became 4294967286 in the submitted run. The safe version rejected the negative value.

• **float to int:** 12.75 became 12 because the fractional part was truncated. The safe version performs the same intentional truncation after checking the range.

• **string to int:** The unsafe program converted the valid string 12345 directly. The safe program was tested with an oversized numeric string and rejected it with an error message.

4. Result

The requested conversions were successfully demonstrated. The vulnerable version showed unexpected and lossy behaviour, while the safe version used range checks and controlled exception handling to reject invalid or out-of-range inputs. This demonstrates that values should be validated before performing potentially unsafe type conversions.

5. Conclusion

The experiment shows that type conversion can cause overflow, sign-conversion issues, truncation, or parsing errors. Safe conversion helpers make the program more predictable by checking the destination range and reporting invalid inputs clearly.